“XMU动物友伴”信息系统

核心理念与产品定位

- **产品名称：** XMU动物友伴 / 厦大动物图鉴
- **核心理念：** 为厦门大学的师生建立一个关于校园内“动物居民”的百科、记录与互动平台。它不仅仅是一个数据库，更是一个承载着同学们与这些动物之间情感与记忆的社区。
- **目标用户：** 全体厦门大学师生。
- **为什么这是个好设计？**
  - **情感驱动：** 学生对猫、狗、天鹅有天然的爱和好奇心。
  - **数据有趣：** 每只动物都是一个独特的“数据个体”，有名字、性格、故事。
  - **功能清晰：** 核心是“记录”与“分享”，逻辑简单。
  - **扩展性强：** 可以从简单的信息展示，逐步增加投喂、健康状态、寻宠等高级功能。

产品设计

用户角色

- **普通用户：** 浏览、点赞、收藏、提交目击记录。
- **动物管理员/核心贡献者：** 审核用户提交的信息，编辑和维护动物档案的权威信息。

核心功能模块

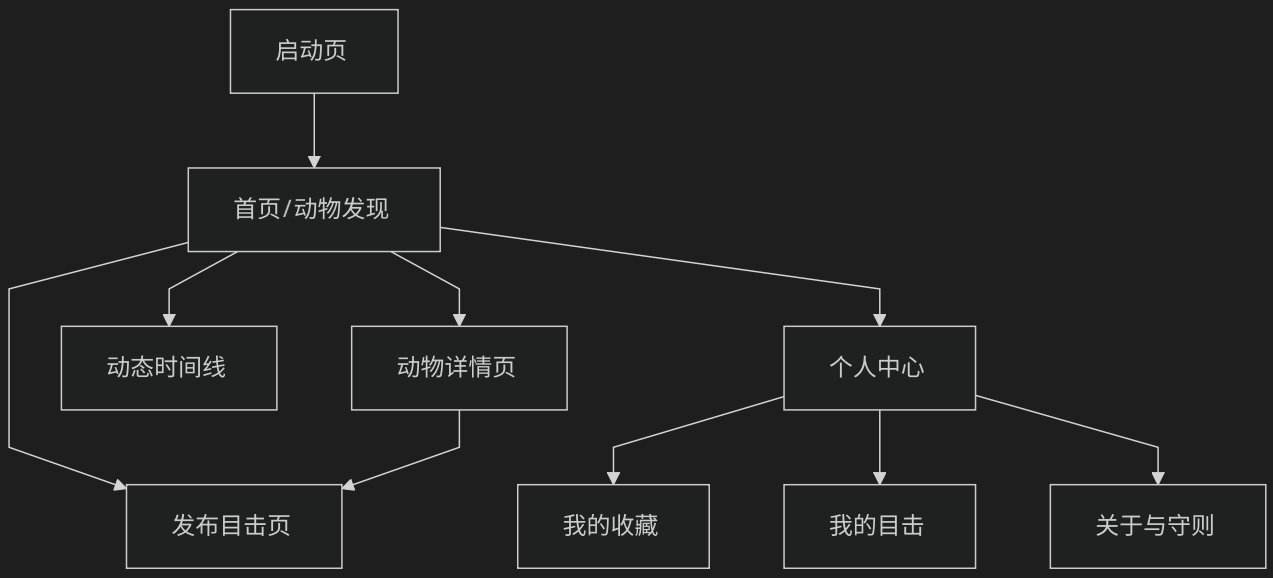
1. **动物档案库**
   - **列表页：** 以卡片形式展示所有动物，可按物种（猫/狗/天鹅）、区域（思明校区/翔安校区、芙蓉隧道/上弦场/芙蓉湖等）、人气、健康状况筛选。
   - **详情页：** 每只动物的“个人主页”。
     - **基本信息：** 官方命名/昵称、物种、性别、绝育情况、常出没地点、性格特点（如“温顺”、“高冷”）。
     - **身份标识：** 最重要的信息！提供多张高清照片，描述外貌特征（如“三花”、“断尾”、“戴白围脖”）。
     - **状态与提醒：** 由管理员标记的健康状态（健康、受伤、待领养等），可设置重要提醒（如“怕狗，请勿靠近”、“正在治疗，请勿投喂”）。
     - **它的故事：** 同学们分享的与它相关的趣事、投稿。
2. **目击与动态**
   - **发布目击：** 用户可像发朋友圈一样，发布在某时某地看到了某只动物（通过选择动物档案关联），并附上照片和文字描述（如“今天在图书馆门口晒太阳，很惬意”）。
   - **动态时间线：** 一个所有用户目击记录的聚合页，可以实时看到校园各处小动物的最新动态。
3. **失联/寻宠协作**
   - 这是一个非常有价值的功能。如果某只常驻的猫咪好几天没出现，用户可以在它的主页标记“疑似失联”或发布“寻宠启事”，发动同学们一起寻找。
4. **科普与守则**
   - 提供科学的流浪动物接触指南、投喂建议、黑天鹅保护须知等，引导学生文明、友善、科学地与小动物相处。

5. **后台管理系统**
    - 动物信息的增删改查、用户目击记录的审核、健康状态更新等。

数据库设计（核心表）

1. `Animals` (动物核心表)
    - `animal_id` (主键)
    - `name` (官方名)
    - `species` (物种：猫/狗/天鹅)
    - `gender` (性别)
    - `is_neutered` (是否绝育)
    - `area` (主要活动区域)
    - `appearance` (外貌特征描述)
    - `personality` (性格描述)
    - `health_status` (健康状态)
    - `avatar_url` (头像图片)
    - `is_missing` (是否失联)
2. `Sightings` (目击记录表)
    - `sighting_id` (主键)
    - `animal_id` (外键，关联Animals)
    - `user_id` (外键，关联用户表)
    - `sighting_time` (目击时间)
    - `sighting_location` (目击地点)
    - `description` (描述)
    - `image_urls` (照片)
    - `status` (状态：待审核/已发布)
3. `Users` (用户表)
    - `user_id` (主键)
    - `username` (用户名，可与学校账号关联)
    - `avatar` (头像)
4. `Missing_Reports` (失联报告表) - *可选高级功能*
    - `report_id`
    - `animal_id` (外键)
    - `reporter_id` (外键)
    - `last_seen_time`
    - `last_seen_location`
    - `report_note`
    - `report_status` (寻找中/已找到)

信息架构与导航



核心页面线框图与功能说明

首页 / 动物发现

- **功能：** 这是用户进入应用后看到的主页面，核心功能是浏览和查找动物。
- **布局：**
  - **顶部搜索栏：** 支持按动物名字或特征搜索。
  - **筛选区：** 提供“全部”、“猫”、“狗”、“天鹅”等标签，以及“区域”下拉筛选。
  - **动物列表：** 以卡片流形式展示。
    - **动物卡片：** 包含动物头像、姓名、常出没区域、性格标签（如“亲人的”、“高冷的”）、健康状态徽章（如“健康”、“受伤”）。

动物详情页

- **功能：** 展示单个动物的完整档案和所有相关动态。
- **布局：**
  - **顶部图集：** 可左右滑动的动物照片。
  - **基本信息区：** 姓名、物种、性别、绝育情况、主要活动区域。
  - **特征与状态：** 详细的外貌描述、性格、以及醒目的健康状态提醒。
  - **“目击/寻宠”按钮组：** 固定悬浮在底部，包含“发布目击”和“报告失联”主要操作按钮。
  - **动态/故事Tab：** 切换查看该动物的所有历史目击记录和用户分享的故事。

发布目击页

- **功能：** 用户提交看到某只动物的记录。
- **布局：**
  - **步骤引导器：** 清晰地显示“选择动物” -> “填写详情” -> “发布成功”。
  - **步骤1 - 选择动物：** 通过列表或在地图上点击位置，关联到一只已有的动物档案。提供一个“未识别/新动物”选项。

    - **步骤2 - 填写详情：**
        - **上传照片：** 必填项。
        - **地点选择：** 调用地图API或手动输入。
        - **时间选择：** 默认当前时间。
        - **状态描述：** 文本框，让用户写下如“看起来很开心”、“好像瘦了”等。
    - **提交按钮**。

动态时间线

- **功能：** 一个所有动物最新目击信息的聚合页面，类似于朋友圈。
- **布局：**
    - **动态卡片：** 每条动态包含用户头像、动物名字、目击时间地点、照片和描述文字。用户可以在此点赞或评论。

个人中心

- **功能：** 管理用户的个人数据和内容。
- **布局：**
    - 用户头像和昵称。
    - **功能入口：** “我的收藏”、“我的目击记录”、“我报告的失联信息”。
    - **通用设置：** “动物守则与科普”、“关于我们”、“设置”。

核心风格定义采用清新简洁的校园风小程序界面，要求柔和色彩搭配大量留白，融入圆角设计和动物元素点缀，以2.5D插画风格呈现。色彩方案围绕厦门大学主题色展开，主色调为嘉庚红搭配象牙白，芙蓉湖蓝作为点缀色，整体营造温暖可信赖的视觉感受，确保符合教育机构调性且具备良好的无障碍色彩对比。

具体页面生成部分包含五个核心界面。首页采用动物信息卡片列表布局，包含顶部搜索栏和分类筛选标签，动物卡片需展示头像、姓名、位置和状态标签。动物详情页需要设计顶部轮播图区域和基本信息卡片，重点突出特征描述段落和健康状态徽章，底部固定操作栏包含发布目击和报告失联按钮。发布目击流程页采用多步骤表单界面，明确展示步骤指示器和动物选择网格布局，集成地图位置选择组件和图片上传区域。动态时间线页面模仿社交动态流界面，采用卡片式设计展示用户头像、动物信息和互动按钮。个人中心页面需要清晰的用户信息卡片和功能入口网格布局，合理分组我的收藏、我的目击、失联报告等功能入口。

视觉元素生成部分专门为图标设计、动物头像占位图和空状态插图提供详细指引。图标系列要求线性设计风格，涵盖猫爪、狗爪、天鹅、地图定位等动物相关主题，保持2px描边和圆角端点的统一设计语言。动物头像生成以可爱的校园动物为主题，采用卡通插画风格展现温和表情，圆形构图融入厦门大学背景元素。空状态插图需要设计没有找到动物的情境，通过疑惑可爱的小女孩形象和问号元素传达友好提示。

实用技巧部分提供质量提升后缀组合，推荐使用quality参数和style raw来优化输出效果，明确排除模糊、水印和文字干扰。一致性控制强调保持同一角色和设计风格，确保色彩方案和组件样式的统一连贯。组件化生成策略建议单独生成导航栏、按钮、输入框等UI组件，便于在设计工具中灵活组合使用。

进阶应用包含交互状态展示和响应式布局展示。交互状态需要生成按钮的默认、悬停、点击、禁用等状态，展现平滑的色彩变化和状态过渡。响应式布局要求展示同一界面在手机、平板、桌面端的不同显示效果，体现布局自适应变化和内容重新排列的逻辑。

三人角色建议（按技能倾向分配）

成员	主要职责	次要/协作职责	关键交付物
成员A：前端工程师	- Vue 3 前端页面开发 - 组件封装与状态管理 - 调用 API 并处理响应 - 地图/图片上传等交互实现	- 参与 UI/UX 讨论 - 编写前端接口文档（Mock 数据） - 协助测试前端功能	- 完整可运行的前端工程 - 动物卡片、详情页、发布流程等页面 - Axios 请求封装
成员B：后端工程师	- Spring Boot 接口开发 - JWT 认证与权限控制 - 业务逻辑实现（审核、失联标记等） - 文件上传至 OSS	- 设计 RESTful API 文档 - 编写单元测试 - 协助数据库表结构调整	- 所有 `/api/*` 接口 - 用户/动物/目击等核心服务 - 全局异常处理机制
成员C：全栈协调 + 数据库/DevOps	- 数据库设计与建表 - SQL 优化与索引设计 - 项目环境搭建（Docker/Nginx） - Git 分支管理与 CI/CD（可选）	- 协助前后端联调 - 编写种子数据（如校区区域、物种枚举） - 部署测试环境	- `schema.sql` / `data.sql` - Docker Compose 配置 - Postman 接口集合 - 部署文档

💡 **提示**：成员C虽偏后端基础设施，但需具备一定全栈视野，是团队“粘合剂”。

具体任务拆解（按模块）

1. 动物档案库

- **前端A**：实现动物列表页（筛选+卡片）、详情页（Tab 切换、悬浮按钮）
- **后端B**：提供 `/animals` 列表接口（支持分页+多条件查询）、详情接口
- **成员C**：设计 `Animals` 表字段，添加索引（如 `species`，`area`），插入初始动物数据

2. 目击与动态

- **前端A**：开发“发布目击”三步表单（选动物→填信息→提交），动态时间线卡片流
- **后端B**：实现目击创建、审核状态变更、时间线聚合查询
- **成员C**：设计 `Sightings` 表，确保外键约束；配置 OSS 图片上传路径

3. 失联/寻宠协作

- **前端A**：在动物详情页增加“报告失联”按钮，跳转表单页
- **后端B**：实现失联报告创建、状态更新（寻找中/已找到）
- **成员C**：创建 `Missing_Reports` 表，编写定时任务脚本（可选：自动标记长期未出现动物）

4. 用户系统与权限

- **后端B**：实现 JWT 登录（模拟或对接厦大认证）、角色判断（USER/ADMIN）
- **前端A**：根据用户角色显示/隐藏“审核”“编辑”按钮
- **成员C**：在 `Users` 表中加入 `role` 字段，默认为 USER；准备管理员账号

5. 后台管理系统（可选 MVP 版）

- **前端A**：用新路由 `/admin` 开发简单管理页（动物列表+审核按钮）
- **后端B**：提供管理员专用接口（需 `@PreAuthorize("hasRole('ADMIN')")`）
- **成员C**：配置 Spring Security 权限规则

协作流程建议

1. 接口先行（Contract First）

- **步骤**：
  1. 成员B 和 成员C 共同制定 **API 文档**（可用 Swagger 或 Markdown）
  2. 文档包含：URL、方法、请求参数、响应结构、错误码
  3. 前端A 根据文档 **Mock 数据开发**（如使用 `vite-plugin-mock`）
  4. 后端完成后，切换真实接口

  优势：前后端并行开发，减少等待

2. Git 分支策略

```
main            # 生产环境稳定版
├── develop     # 集成分支（每日合并）
├── feat/animal-list      # 前端A
├── feat/sighting-api     # 后端B
└── infra/db-setup        # 成员C
```

- 每人从 `develop` 拉特性分支
- 完成后提 PR → 至少一人 Code Review → 合并回 `develop`

3. 联调会议（每周 1~2 次）

- 检查进度：谁卡住了？接口是否对齐？
- 演示功能：前端展示页面，后端演示 Postman 调用
- 调整计划：优先级变更、范围收缩（MVP 原则）

共享资产清单（避免重复劳动）

资产	负责人	存放位置
API 接口文档	成员B	`/docs/api.md` 或 Swagger UI
数据库 ER 图	成员C	`/docs/db-er.png`
UI 设计稿/配色	全体	Figma / 或直接用 Element Plus 默认主题
环境配置说明	成员C	`/README.md` + `.env.example`
Postman 集合	成员C	导出 JSON 放入 `/docs/postman.json`

注意事项

1. **不要严格割裂**：前端可帮写简单 SQL 查询，后端可调样式 bug。
2. **统一命名规范**：
    - 前端：`camelCase`（如 `animalId`）
    - 后端：Java 字段 `camelCase`，数据库 `snake_case`（如 `animal_id`）
3. **尽早跑通“最小闭环”**：
    - 第一天目标：用户能登录 → 看到一只猫 → 提交一次目击 → 后台看到记录
4. **日志与错误处理**：
    - 后端记录关键操作日志（如“用户123提交目击”）
    - 前端友好提示（如“网络错误，请重试”而非白屏）